algorithm is used for path navigation but also in AI and game theory.

---

[1] An algebraic data type is one particular kind of composite type, formed by combining other types, usually with a definition “by induction”, with one or more base types, and operators to combine them.
This presentation nicely explains how to define them in C++ and what are the benefits: https://www.youtube.com/watch?v=ojZbFIQSdl8

[2] For an example with Haskell, see: Mokhov, Andrey. "Algebraic graphs with class (functional pearl)." ACM SIGPLAN Notices. Vol. 52. No. 10. ACM, 2017. A Scala implementation (ongoing, at the time of writing) can be found here: https://github.com/algebraic-graphs/scala

[3] Any directed graph where there is an edge (u,v) but not the inverse edge (v,u) can’t be transformed into an isomorphic undirected graph (meaning, an undirected graph with the same shape, i.e. with the same set of vertices, connected in the same way).

[4] Have you ever heard of the “traveling salesman problem”, or TSP? That’s one of the most studied hard problems.

[5] By either removing the checks for goal, or passing a function that always returns `false` as the `isGoal` argument.

[6] Just to be clear, this is different than computing the optimal route through several, or all, vertices: single-source-all-destinations only computes the optimal paths from the source to each other vertex taken individually.

[7] The Manhattan distance, also known as block distance, is the sum of the absolute difference of the Cartesian coordinates of two points. The name comes from the fact that in the island of Manhattan, most streets have a grid layout, and so the shortest path between two intersections has distance equal to the sum of the block’s sides.

[8] Please do **NOT** *literally* hold your breath! And no, not even if you are on speed reading.

[9] We’ll use a min-heap to store vertices in the frontier, and their distance from source is used as priority.

[10] If a graph has a negative-weight cycle, defined as a cycle for which the sum of edges’ weights is negative, then discussing shortest-paths could be meaningless: by repeatedly traversing the cycle, in fact, one could get to an arbitrarily low total cost.

[11] Of course, using patterns like template or strategy will help keep your codebase DRY and easier to maintain, but the point is, also implementing general purpose methods or classes will make your code less clean and maintainable, so you need to balance out both aspects.

[12] Completeness guarantees that only a finite number of nodes will have to be visited before reaching the goal.

[13] For trees, admissibility alone is a sufficient condition: can you explain why? Hint: in a tree, how many paths from root to a goal vertex pass through a given node u?